

Week 10 — Implementing a 1D finite element code

From the lecture notes to Julia: mesh, reference element, geometrical map, FE space, assembly

MTH3340 · Numerical Methods for PDEs

1 The target

Compute $\mathbf{A}$ and $\mathbf{b}$ in the finite element system for the 1D Poisson problem

$$-(\kappa u')' = f \text{ in } \Omega = (a, b), \quad u(a) = u_a, \quad u(b) = u_b,$$

with Lagrangian finite elements of **arbitrary order p** . Weak form + Galerkin:

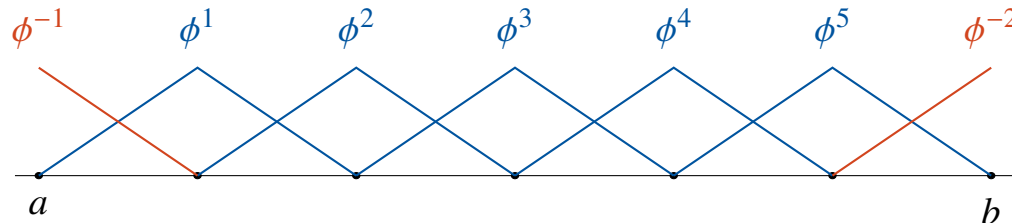
$$u_h \in V_h^D : \quad \int_{\Omega} \kappa u'_h v'_h = \int_{\Omega} f v_h \quad \forall v_h \in V_h^0.$$

V_h : continuous, $\mathcal{P}_p$ on each cell. V_h^D : $v(a) = u_a, v(b) = u_b$. V_h^0 : $v(a) = v(b) = 0$.

We never need V_h itself: only the trial space V_h^D (affine) and the test space V_h^0 (vector space). Both are described by the *same* nodes.

The deliverable is a program. The plan (1d-code-plan.jl) asks for *structs* with constructors and methods, one per ingredient of Chapters 2–3. Today: what each struct must know, and why.

2 Free and fixed degrees of freedom



One hat function per node. Two kinds of nodes:

- **free** DOFs: interior nodes, **positive** global indices $1, 2, \dots$;
- **fixed** DOFs: the nodes at a and b , **negative** indices $-1, -2$.

In V_h^D the fixed DOFs hold u_a, u_b ; in V_h^0 they are 0. The free ones are the unknowns.

One set of nodes, one numbering, two spaces.

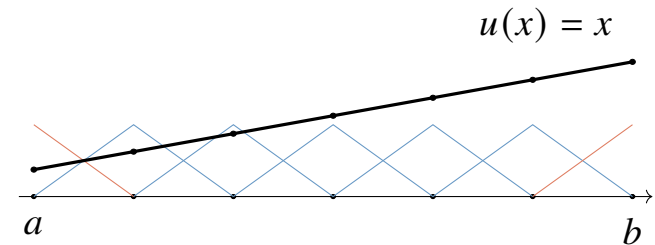
3 A function is two vectors

Any $v \in V_h$ on this mesh is determined by **7 numbers** (its nodal values), split as

$$\vec{u}_{\text{fixed}} \in \mathbb{R}^2, \quad \vec{u}_{\text{free}} \in \mathbb{R}^5.$$

E.g. $u(x) = x$ with $h = (b - a)/6$:

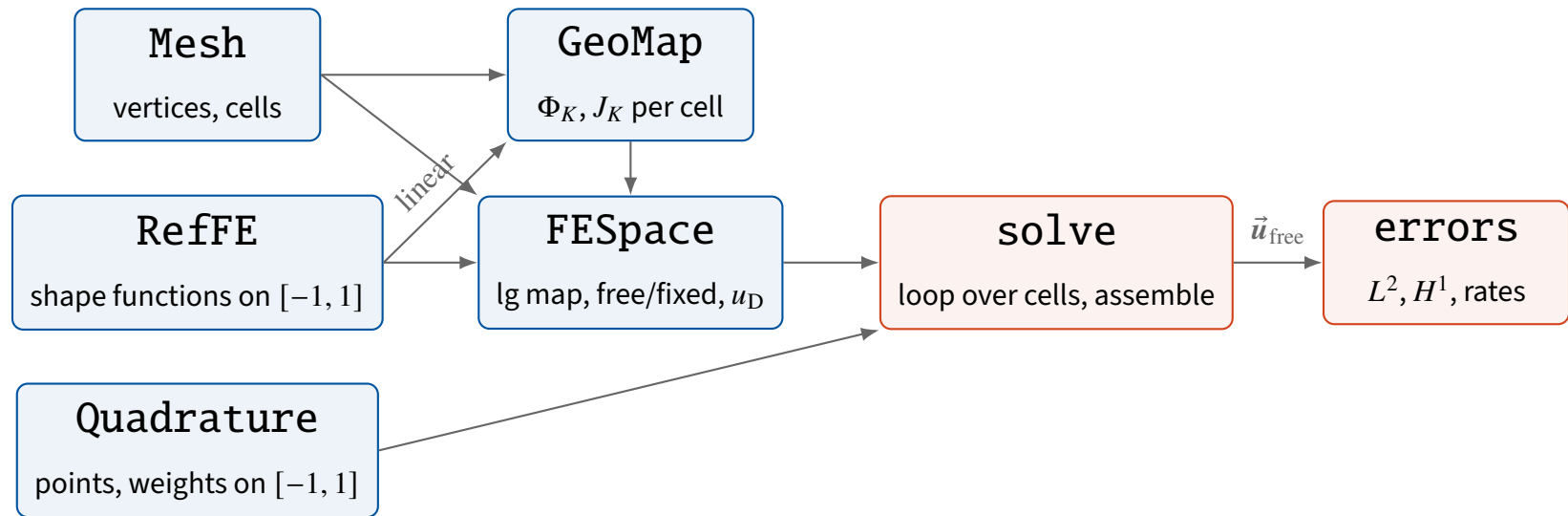
$$\vec{u}_{\text{fixed}} = \begin{bmatrix} a \\ b \end{bmatrix}, \quad \vec{u}_{\text{free}} = \begin{bmatrix} a + h \\ \vdots \\ a + 5h \end{bmatrix}.$$



From the function to the vectors: **interpolation** = evaluate the DOFs = **evaluate the function at the nodes**.

So the code must compute the **node locations** and **label each node** free or fixed.

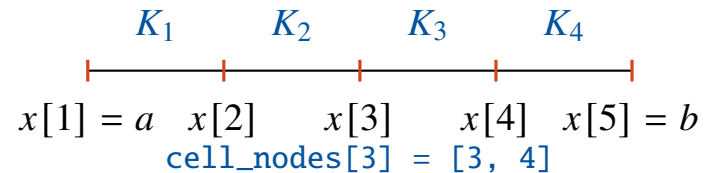
4 The architecture of the code



- Each ingredient of Chapter 3 becomes a **type**: a constructor that does the maths, and a few **methods** that return its data (`get_cell_nodes`, `get_weights`, ...).
- **Keep the names of the maths**: the code should read like Sections 3.3–3.9.
- The blue boxes are independent: **write and test them one at a time**.

5 Mesh: geometry only

A uniform partition of (a, b) into N cells is described by the **vertex coordinates** $x[1] = a < \dots < x[N+1] = b$ and, for each cell, the **global ids of its two vertices**.



```
struct Mesh
    nodes_coordinates
    cell_nodes
```

```
end
```

```
Mesh(a, b, N)    # uniform mesh
```

```
# methods (the API of the struct)
```

```
get_node_coordinates(m::Mesh) = m.nodes_coordinates
```

```
get_cell_nodes(m::Mesh) = m.cell_nodes
```

```
num_cells(m::Mesh) = length(m.cell_nodes)
```

Hints. `range(a, b, length = N+1)`; `cell_nodes` as a vector of 2-vectors. The rest of the code only uses the methods, never the fields.

Q: Should the extra nodes of an order- p space live in the mesh? (Geometry versus approximation.)

6 Quadrature on the reference cell

Everything is integrated on $\hat{K} = [-1, 1]$ (Section 3.9) with a rule

$$\int_{-1}^1 g(\hat{x}) \, d\hat{x} \approx \sum_{\text{gp}} g(\hat{x}_{\text{gp}}) w_{\text{gp}}.$$

```
struct Quadrature
  points
  weights
end
Quadrature(degree) # exact up to degree
```

```
# methods
get_integration_points(q::Quadrature) = q.points
get_weights(q::Quadrature) = q.weights
num_points(q::Quadrature) = length(q.points)
```

Hint. Do not compute Gauss points yourself: with `FastGaussQuadrature`, `gausslegendre(m)` gives the m points and weights on $[-1, 1]$.

Q: The m -point Gauss rule is exact up to degree $2m - 1$. How many points do you need for a given degree?

Test: $\int_{-1}^1 \hat{x}^2 = 2/3$, $\int_{-1}^1 \hat{x}^3 = 0$, $\sum_{\text{gp}} w_{\text{gp}} = 2$.

7 The reference finite element

Order p on $\hat{K} = [-1, 1]$: $p + 1$ equispaced nodes $\hat{x}_1 = -1 < \dots < \hat{x}_{p+1} = 1$ and shape functions defined by $\hat{\phi}^i(\hat{x}_j) = \delta_{ij}$ (Section 3.5). We use the [Lagrange polynomials](#) and their derivatives (product rule):

$$\hat{\phi}^i(\hat{x}) = \prod_{j \neq i} \frac{\hat{x} - \hat{x}_j}{\hat{x}_i - \hat{x}_j}, \quad (\hat{\phi}^i)'(\hat{x}) = \sum_{k \neq i} \frac{1}{\hat{x}_i - \hat{x}_k} \prod_{j \neq i, k} \frac{\hat{x} - \hat{x}_j}{\hat{x}_i - \hat{x}_j}.$$

(A monomial pre-basis + change of basis, Section 3.5, is more general but more work.)

```
struct RefFE
  shape_functions::Vector{Function}
  gradient_shape_functions::Vector{Function}
end
RefFE(p)  # order p, equispaced nodes
```

```
# methods
get_shape_functions(r::RefFE) = r.shape_functions
get_gradient_shape_functions(r::RefFE) =
  r.gradient_shape_functions
num_dofs(r::RefFE) = length(r.shape_functions)
```

The struct stores a [vector of functions](#), $[\hat{\phi}^1, \dots, \hat{\phi}^{p+1}]$, and one of their derivatives. [Hint](#). A function can return a function: `lagrange(i) = x -> ...` is the i -th shape function. [Test](#): $\hat{\phi}^i(\hat{x}_j) = \delta_{ij}$, $\sum_i \hat{\phi}^i \equiv 1$.

8 The geometrical map

$\Phi_K : \hat{K} \rightarrow K = [x_K^1, x_K^2]$, built from the **linear** reference element and the vertex coordinates (Section 3.3):

$$\Phi_K(\hat{x}) = \sum_{i=1}^2 x_K^i \hat{\phi}^i(\hat{x}), \quad J_K(\hat{x}) = \sum_{i=1}^2 x_K^i (\hat{\phi}^i)'(\hat{x}).$$

```
struct GeoMap
  maps::Vector{Function}      # per cell
  jacobian::Vector{Function}  # per cell
end
GeoMap(mesh)    # uses RefFE(1)
```

```
# methods
get_cell_map(g::GeoMap) = g.maps
get_cell_jacobian(g::GeoMap) = g.jacobian
```

Again **vectors of functions**, one per cell: `maps[K]` is the **linear combination of the two linear shape functions** with the vertex coordinates of K as coefficients; the Jacobian likewise, with their derivatives.

Q: What is J_K for a straight 1D cell? Then why program the general formula?

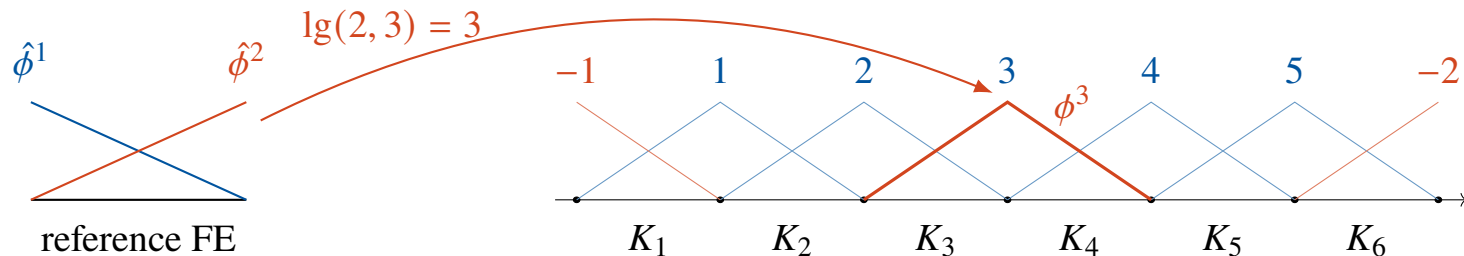
Test: $\Phi_K(-1) = x_K^1$, $\Phi_K(1) = x_K^2$, $\sum_K \int_{\hat{K}} |J_K| = b - a$.

9 From local to global

Global shape functions are glueded from local ones (Section 3.6) through the **local-to-global map**: for a local node i of the reference element and a cell K ,

$$\phi^a(x)|_K = \hat{\phi}^i(\Phi_K^{-1}(x)), \quad a = \text{lg}(i, K).$$

Two local nodes are the same global node iff they sit at the **same point** of Ω .
We just store this table; ϕ^a is never written explicitly.



10 The lg table

For $p = 1$ and the 6-cell mesh:

$$\lg(\cdot, 3) = [2, 3], \quad \lg(\cdot, 6) = [5, -2].$$

One row per cell (a matrix, or a vector of vectors); for $p > 1$, **more columns**.

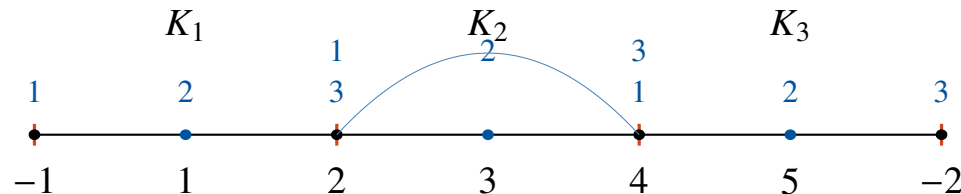
Negative entries are the fixed nodes, with the sign convention of board 2.

Implement this in your FESpace!

Careful: a vertex shared by two cells must receive the *same* global id from both (that is the whole point). No node coordinates are needed.

$$\lg = \begin{bmatrix} -1 & 1 \\ 1 & 2 \\ 2 & 3 \\ 3 & 4 \\ 4 & 5 \\ 5 & -2 \end{bmatrix}$$

11 Global nodes for $p > 1$



$p = 2, N = 3$: local node numbers in blue, DOF ids below. Cell K owns the $p + 1$ global nodes $(K - 1)p + 1, \dots, (K - 1)p + p + 1$.

- consecutive cells **share the interface node**: this is what makes $V_h \subset C^0$, hence H^1 -conforming (Section 3.6);
- $Np + 1$ nodes; the **two end nodes are fixed**, the other $Np - 1$ are free.

This closed-form numbering is **one option**, possible because the mesh is a 1D line. The general alternative: sweep the cells and **number the nodes as you meet them**, cell by cell, keeping track of which vertices already have an id so that a shared vertex is not numbered twice.

12 The FESpace struct

What the space must know:

- mesh and reference element;
- the `lg map`;
- number of free / fixed DOFs;
- values at the fixed nodes.

```
struct FESpace
    mesh::Mesh
    reffe::RefFE
    node_map      # lg map
    fixed_values  # u_D there
end
FESpace(mesh, reffe, uD)
# methods
get_cell_dofs(V::FESpace) = V.node_map
get_fixed_values(V::FESpace) = V.fixed_values
num_free_dofs(V::FESpace) = ...
```

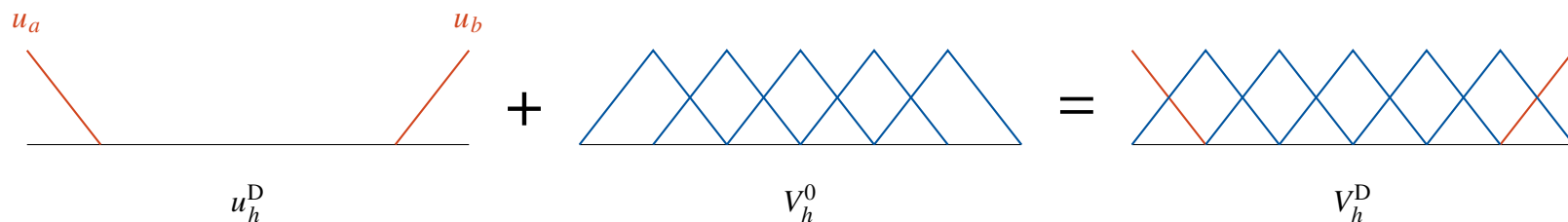
The constructor gets a function `uD` with $u_D(a) = u_a$, $u_D(b) = u_b$, and evaluates it at a and b . $\vec{u}_{\text{free}}$ is *not* stored: a function is (space, vector).

Q: V_h^D and V_h^0 differ only in `fixed_values` (u_a, u_b versus 0, 0). Do you need two objects? What does the solver use from the test space?

13 The offset function method, discretely

Definition 2.3.3: $u_h = u_h^D + u_h^0$ with $u_h^D \in V_h^D$, $u_h^0 \in V_h^0$, and we pick

$$u_h^D \doteq \sum_{k \in \mathcal{N}_{\text{fixed}}} \vec{u}_{\text{fixed},k} \phi^{-k} \quad (\mathcal{N}_{\text{fixed}} = \{1, 2\}).$$



The problem becomes **square**: find $u_h^0 \in V_h^0$ such that

$$\int_{\Omega} \kappa (u_h^0)' v' = \int_{\Omega} f v - \int_{\Omega} \kappa (u_h^D)' v' \quad \forall v \in V_h^0.$$

Trial and test space are now the same vector space.

14 The linear system

Expand the unknown in the free basis and test with every free basis function (bilinearity):

$$u_h^0 = \sum_{j \in \mathcal{N}_{\text{free}}} \vec{u}_{\text{free},j} \phi^j, \quad v = \phi^i, \quad i \in \mathcal{N}_{\text{free}}.$$

$$\boxed{\mathbf{A} \vec{u}_{\text{free}} = \mathbf{b}} \quad \mathbf{A}_{ij} = \int_{\Omega} \kappa (\phi^j)' (\phi^i)',$$

$$\mathbf{b}_i = \int_{\Omega} f \phi^i - \sum_{k \in \mathcal{N}_{\text{fixed}}} \vec{u}_{\text{fixed},k} \int_{\Omega} \kappa (\phi^{-k})' (\phi^i)',$$

for $i, j \in \mathcal{N}_{\text{free}}$.

- $\mathbf{A}$ is $N_{\text{free}} \times N_{\text{free}}$, symmetric, positive definite (Chapter 2);
- the **offset term** in $\mathbf{b}$ involves the *same* integrals as $\mathbf{A}$, with a fixed index $-k$ in the column slot;
- $\mathbf{A}_{ij} \neq 0$ only if ϕ^i and ϕ^j overlap: **sparse**.

15 Cell-wise computation and assembly

We never loop over pairs (i, j) : we **loop over cells**, compute the local matrix (Section 3.8)

$$\mathbf{A}_{\alpha\beta}^K = \int_K \kappa (\hat{\phi}^\beta \circ \Phi_K^{-1})' (\hat{\phi}^\alpha \circ \Phi_K^{-1})', \quad \alpha, \beta = 1, \dots, p+1,$$

and **apply lg to its indices**:

$$\mathbf{A}[\text{lg}(\alpha, K), \text{lg}(\beta, K)] += \mathbf{A}_{\alpha\beta}^K, \quad \text{e.g. } \mathbf{A}^K \in \mathbb{R}^{2 \times 2} \rightarrow \mathbf{A}(3:4, 3:4).$$

Q: What should the assembly do when $\text{lg}(\beta, K) < 0$? That column multiplies a *known* value — look at $\mathbf{b}_i$ on the previous board. And when $\text{lg}(\alpha, K) < 0$?

Consequence. The “offset function term” of the plan needs **no extra loop and no global vector**: a few entries of $\mathbf{A}^K$ move to $\mathbf{b}^K$, only in cells touching the boundary. This is why the FE offset function keeps sparsity (Section 2.3).

16 Integrating the cell matrix

Change of variables to $\hat{K}$ and quadrature (Section 3.9); in 1D, $\phi' = \hat{\phi}'/J_K$:

$$\begin{aligned} \mathbf{A}_{\alpha\beta}^K &= \int_{\hat{K}} \kappa \circ \Phi_K \frac{(\hat{\phi}^\beta)'}{J_K} \frac{(\hat{\phi}^\alpha)'}{J_K} |J_K| d\hat{x} \\ &\approx \sum_{\text{gp}} \kappa(\Phi_K(\hat{x}_{\text{gp}})) \frac{(\hat{\phi}^\beta)'(\hat{x}_{\text{gp}})}{J_K(\hat{x}_{\text{gp}})} \frac{(\hat{\phi}^\alpha)'(\hat{x}_{\text{gp}})}{J_K(\hat{x}_{\text{gp}})} |J_K(\hat{x}_{\text{gp}})| w_{\text{gp}}, \end{aligned}$$

and $\mathbf{b}_\alpha^K \approx \sum_{\text{gp}} f(\Phi_K(\hat{x}_{\text{gp}})) \hat{\phi}^\alpha(\hat{x}_{\text{gp}}) |J_K(\hat{x}_{\text{gp}})| w_{\text{gp}}$.

```
for K in cells                # A_K, b_K = 0
    for gp in quadrature points # x = Phi_K(xhat), J = J_K(xhat)
        for i, j in local DOFs
            A_K[i,j] += ...    # idem b_K[i] (forcing, offset)
        assemble A_K, b_K into A, b using lg(:,K)
    u_free = A \ b
```

Hint. $\hat{\phi}^\alpha(\hat{x}_{\text{gp}})$, $(\hat{\phi}^\alpha)'(\hat{x}_{\text{gp}})$ do not depend on K : **tabulate them once.**

Q: Which degree do the entries of $\mathbf{A}^K$ have for constant κ and order p ? And those of $\mathbf{b}^K$? Choose degree accordingly.

17 The finite element function

Solving $\mathbf{A} \vec{u}_{\text{free}} = \mathbf{b}$ ($u_{\text{free}} = \mathbf{A} \setminus \mathbf{b}$) gives **only the free values**. To evaluate u_h (errors, plots) we need a function of V_h .

A function of V_h is a pair: the **space** plus the **vector of free values** — the fixed values already live in the space, and so does the lg map. Again a **vector of functions**, one per cell:

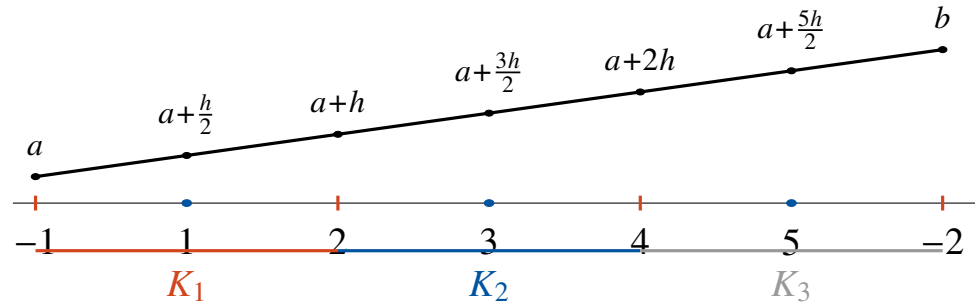
$$u_h|_K(\hat{x}) = \sum_{\alpha=1}^{p+1} u_{\text{lg}(\alpha,K)} \hat{\phi}^\alpha(\hat{x}), \quad u_a = \begin{cases} \vec{u}_{\text{free}}[a] & a > 0, \\ \vec{u}_{\text{fixed}}[-a] & a < 0. \end{cases}$$

```
struct FEFun
  space::FESpace
  free_values
end
```

```
# methods, one entry per cell
get_cell_values(uh)      # [u_lg(1,K), ..., u_lg(p+1,K)]
get_cell_functions(uh)  # xhat -> the sum above
# and, for plots or point errors,
evaluate(uh, points)    # u_h at physical points x
```

Hints. `get_cell_values` is the only place where the sign of an lg entry matters. For `evaluate`: find the cell K containing x (easy on a uniform mesh), pull x back to $\hat{x} = \Phi_K^{-1}(x)$ (a linear map: invert it by hand) and call the cell function.

18 Example: $u(x) = x$ with quadratic elements



$p = 2, N = 3, h = (b - a)/3$; DOF ids below the axis, nodal values above:

$$\vec{u}_{\text{fixed}} = [a, b], \quad \vec{u}_{\text{free}} = [a + \frac{h}{2}, a + h, a + \frac{3h}{2}, a + 2h, a + \frac{5h}{2}].$$

K_1 (touches the Dirichlet boundary), $\text{lg}(\cdot, 1) = [-1, 1, 2]$:

$$\text{cell values} = [\vec{u}_{\text{fixed}}[1], \vec{u}_{\text{free}}[1], \vec{u}_{\text{free}}[2]] = [a, a + \frac{h}{2}, a + h].$$

K_2 (interior), $\text{lg}(\cdot, 2) = [2, 3, 4]$:

$$\text{cell values} = [\vec{u}_{\text{free}}[2], \vec{u}_{\text{free}}[3], \vec{u}_{\text{free}}[4]] = [a + h, a + \frac{3h}{2}, a + 2h].$$

Each $\sum_{\alpha} (\text{cell value})_{\alpha} \hat{\phi}^{\alpha} \circ \Phi_K^{-1}$ reproduces x exactly: a first test.

19 Verification

Manufactured solutions. Choose u ; set $f \doteq -(\kappa u')'$ and $u_D \doteq u$ at a, b . Then u is the exact solution and $u - u_h$ can be measured.

1. **Machine precision.** u polynomial of degree $\leq p$, so $u \in V_h$.

Q: Which error must you get, and why? (Galerkin orthogonality, Céa.)

2. **Convergence.** $u = \sin(\pi x)$ on $(0, 1)$, halve h repeatedly. Chapter 2:
 $\|u - u_h\|_{L^2} \sim h^{p+1}$, $|u - u_h|_{H^1} \sim h^p$. Rate = $\log(e_1/e_2)/\log(h_1/h_2)$.
3. **Non-homogeneous data, variable κ :** e.g. $u = x^3$, $\kappa = 1 + x$.

Errors cell by cell, with the cell functions of the FEFunctor and your Quadrature:

$$\|u - u_h\|_{L^2}^2 = \sum_K \int_K (u - u_h)^2 = \sum_K \int_{\hat{K}} (u \circ \Phi_K - u_h|_K)^2 |J_K| \, d\hat{x}.$$

Q: Can you reuse the assembly quadrature here and call the result exact?

20 Plan of work

Build bottom-up and **test each struct before using it in the next**:

1. Mesh: print it for $N = 4$; cell lengths add up to $b - a$.
2. Quadrature: $\int_{-1}^1 \hat{x}^2 = 2/3$; weights sum to 2.
3. RefFE: $\hat{\phi}^i(\hat{x}_j) = \delta_{ij}$; partition of unity.
4. GeoMap: $\Phi_K(\pm 1)$ are the vertices; $J_K = h_K/2$.
5. FESpace: print the lg table for $p = 1, 2$, $N = 3$; shared vertices must agree.
6. FEFunction and solver with $f = 0$, linear u_D : u_h must be that line (boards 3, 18).
7. Manufactured solutions: machine precision, then convergence rates.

Start with $p = 1$, $N = 2$: one free DOF, a 1×1 system you can check by hand (Section 2.3). Only then let p and N grow.